



TECHNICAL SOLUTION DOCUMENT

ARA Lead Matrix CRM Platform

Solution, Data, API, Security & Infrastructure Architecture

The complete technical implementation of the platform — solution & logical architecture, the MongoDB data model, the full REST API catalogue, authentication & security design, background processing, integrations, deployment topology, and the strategies for logging, caching, scalability, availability and disaster recovery.

PREPARED FOR

Architects · Developers · DevOps ·
QA

PREPARED BY

Solution Architecture Office

VERSION

1.0 (Baseline)

DATE OF ISSUE

17 July 2026

CLASSIFICATION

Confidential

COMPANION TO

BRD-2026-001 · FSD-2026-001

Governance and configuration of this Technical Solution Document (TSD).

Attribute	Detail
Document Title	ARA Lead Matrix CRM — Technical Solution Document
Reference	ARA-TSD-CRM-2026-001
System	ARA Lead Matrix CRM ("New Ara CRM")
Companion Documents	BRD (ARA-BRD-CRM-2026-001) · FSD (ARA-FSD-CRM-2026-001)
Version / Status	1.0 — Baseline — Approved for Review
Classification	Confidential
Date of Issue	17 July 2026
Codebase baseline	Backend ≈ 21,250 LOC (Node/Express ESM); React SPA; 24 Mongoose models; 18 route groups.

Intended Audience

- **Solution / Software Architects** — architecture, data model, integration design.
- **Developers** — module & component structure, API contracts, coding standards.
- **DevOps Engineers** — deployment, infrastructure, configuration, CI/CD, DR.
- **QA Engineers** — testing strategy, validation, error handling.
- **Enterprise Clients** — technical assurance of the delivered solution.

Approval & Sign-Off

Role	Responsibility	Signature	Date
Solution Architect	Architecture accuracy		
Engineering Lead	Implementation fidelity		
DevOps Lead	Infra & deployment accuracy		
Security Officer	Security posture review		

Basis. Produced from a full source-level review of the delivered repository (backend services, models, routes, middleware, background jobs, utilities, front-end, build & deploy scripts and environment configuration). It describes the system *as built and in production* as of the issue date.

Revision History

Version	Date	Author	Summary
0.1	12 Jul 2026	Solution Architecture Office	Architecture outline & section structure.
0.6	15 Jul 2026	Solution Architecture Office	Data, API, security & integration architecture drafted from source.
0.9	16 Jul 2026	Solution Architecture Office	Deployment, DR, diagrams, risks & recommendations added.
1.0	17 Jul 2026	Solution Architecture Office	Baseline — full 54-section TSD approved.

Table of Contents

GOVERNANCE

1	Cover Page
2	Document Control
3	Revision History
4	Table of Contents

ARCHITECTURE

5	Exec. Technical Summary
6	Architecture Overview
7	Solution Architecture
8	Logical Architecture
9	Physical Architecture
10	Deployment Architecture
11	Infrastructure Overview
12	Technology Stack

STRUCTURE

13	Project Structure
14	Folder Structure
15	Module Architecture
16	Component Architecture
17	Layered Architecture

DATA

18	Database Architecture
19	ER Diagrams
20	Database Tables
21	Database Relationships

API & SECURITY

22	API Architecture
23	API Catalogue
24	Authentication Arch.
25	Authorization Arch.
26	Security Architecture

OPERATIONS

27	Logging Strategy
28	Monitoring Strategy
29	Error Handling
30	Validation Strategy
31	Performance Optim.
32	Caching Strategy
33	Scalability Strategy
34	High Availability
35	Backup Strategy
36	Disaster Recovery

DELIVERY

37	CI/CD Overview
38	Environment Config
39	Configuration Files
40	Third-party Integr.

DIAGRAMS

41	Sequence Diagrams
42	Data Flow Diagrams
43	Component Diagrams
44	Class Diagrams
45	Package Diagrams
46	Deployment Diagrams
47	Module Dependencies

ASSURANCE

48	Coding Standards
49	Testing Strategy
50	Security Considerations
51	Technical Risks
52	Future Scalability
53	Recommendations
54	Appendix

ARA Lead Matrix is a multi-tenant MERN-stack CRM: a React single-page front-end, a stateless Node.js/Express REST API, a MongoDB (Azure Cosmos DB for MongoDB vCore) datastore, and an in-process background-job engine that synchronises Meta and Google Ads data and reconciles advertising spend into an immutable billing ledger.

MERN ARCHITECTURE PATTERN	24 COLLECTIONS	18 API ROUTE GROUPS	3 EXTERNAL INTEGRATIONS	2 AUTH REALMS
--	--------------------------	-------------------------------	--------------------------------------	-------------------------

5.1 Architectural characteristics

Characteristic	Implementation
Statelessness	JWT-based sessions; no server session store — the API scales horizontally behind a load balancer without sticky sessions (subject to the single-instance scheduler caveat, §33).
Multi-tenancy	Row-level tenancy on a shared schema; client identity for portal tokens is derived from the JWT and enforced per request.
Idempotency	Unique/compound indexes + idempotency keys guarantee exactly-once lead ingestion and billing writes.
Resilience	Typed integration errors, exponential-backoff retries, a webhook dead-letter queue, overlap guards, and boot-time self-healing migrations.
Separation of concerns	Routes → Controllers → Services → Models, with a dedicated single metrics service as the canonical KPI authority.
Security at rest	bcrypt for passwords; AES-256-CBC for recoverable secrets (page tokens, vault, portal-password recovery).

5.2 Key technical decisions & rationale

Decision	Rationale
Single-database, shared-schema multi-tenancy	Simplicity + cross-client agency reporting; tenancy enforced at the query layer.
In-process cron scheduler (node-cron)	Zero extra infrastructure; adequate for current client volume; must run on exactly one instance (§33).
Meta insights as reporting source of truth	Avoids under-reporting when lead ingestion is briefly degraded.
Delta-based idempotent billing ledger	Financial correctness under retries and concurrency.
Post-build JS obfuscation	Raises the bar against trivial client-code inspection (identifier mangling; heavy transforms deliberately disabled for runtime safety).
Cosmos DB vCore with small connection pools	Respects a hard per-cluster connection ceiling shared across processes.

A classic three-tier web architecture augmented with an integration/background-processing tier.

HIGH-LEVEL ARCHITECTURE

CLIENT TIER — BROWSERS

Agency Console (React SPA)

Client Portal (React SPA)

▼ HTTPS / JWT

PRESENTATION / EDGE — NGINX

Static asset host (/var/www/leadmatrix)

Reverse proxy → :5000

TLS termination



APPLICATION TIER — NODE.JS / EXPRESS (SYSTEMD: ARA-CRM)

Routes / Middleware

Controllers

Services

Single Metrics Service

Background jobs — node-cron (Google sync · Meta sync ·
retry worker)

Webhook receiver (Meta leadgen)

▼ Mongoose (pool ≤5)

DATA TIER

MongoDB — Azure Cosmos DB for MongoDB vCore (24 collections)

◀▶ outbound API

EXTERNAL SYSTEMS

Meta Graph API

Google Ads API

Google Gemini

6.1 Architectural styles applied

- **Three-tier** (presentation / application / data) with a **REST** API contract.
- **Layered** application internals (routes → controllers → services → models).
- **Event-driven ingestion** for Meta leads (webhook push) with **scheduled batch** reconciliation.
- **Shared-schema multi-tenant SaaS**.

How the business capability is realised technically end-to-end.

Capability	Technical realisation	Key components
Identity	Two JWT realms (staff, portal-user); bcrypt credentials; cookie + bearer transport.	authController, clientAuthController, auth middleware, generateToken
Ad data acquisition	Scheduled per-client pulls + real-time webhooks; typed errors; rate-limit awareness.	scheduler, syncService, metaSyncService, metaAdsService, googleAdsService
Lead ingestion	Signed webhook → raw audit → idempotent upsert → retry queue → poller safety-net.	metaController, metaLeadService, MetaLeadRaw, MetaWebhookRetry
Telecalling	26-column sheet; pre-save automation library; Excel round-trip.	leadTelecallerController, telecallerAutomation, leadXlsxService
Billing	Delta reconcile vs snapshot; immutable idempotent ledger; balance cache.	metaBillingService, syncService, BillingTransaction, DailyDebitSnapshot
Reporting	One metrics service feeds all surfaces; short-TTL caches.	metaMetricsService, analyticsService, reportsController
Intelligence	Gemini REST call with strict-JSON contract + repair.	aiController
Presentation	Lazy-loaded React SPA; Redux slices + context caches; MUI.	App.js, store, contexts, pages, components

7.1 Cross-cutting concerns

Concern	Mechanism
Security	helmet, CORS allow-list, two-tier rate limiting, JWT verify, tenant scoping, encryption utilities.
Consistency	Canonical metrics service; shared enum constants (front/back in lockstep).
Observability	MetaSyncRun log, retry queue, raw-lead audit, morgan HTTP logs, tagged diagnostics.
Data integrity	Unique/compound/partial indexes; idempotency keys; boot-time self-heal.

LOGICAL LAYERS

L1 • PRESENTATION (REACT)

Pages / Layouts

Components

Redux slices & Contexts

API clients (axios)

▼ REST/JSON

L2 • API & MIDDLEWARE

Route groups (/api/*)

Auth & RBAC guards

Validation

Rate limit / CORS /
helmet

Error handler

L3 • DOMAIN / CONTROLLERS

Auth

Client

Lead / Telecaller

Meta

Billing

Reports / AI

L4 • SERVICES / INTEGRATION

Meta services

Google Ads
service

Metrics service

Billing services

Xlsx service

Sync + scheduler

L5 • DATA ACCESS (MONGOOSE ODM)

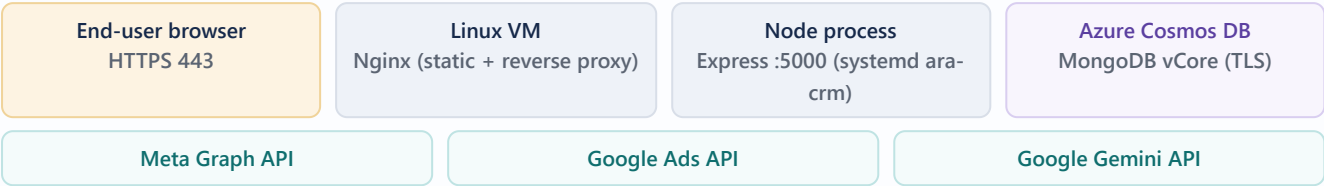
Models / Schemas / Indexes / Hooks / Virtuals

Each layer depends only on the layer beneath it. Controllers never talk to external APIs directly — they delegate to services; services own all Graph/Ads API traffic and normalisation.

8.1 Front-end logical state

Store / context	Responsibility
auth slice	Staff session, role, permissions; persisted to localStorage.
clients / leads / dailyEntry / dailyLeadData slices	Entity lists, CRUD, stats.
DataCacheContext	Caches slow-changing client list across pages.
ThemeContext	Brand colours / dark mode.

PHYSICAL NODES (PRODUCTION)



Node	Detail
Web/edge	Nginx serves the built SPA from <code>/var/www/leadmatrix</code> and reverse-proxies <code>/api</code> to the Node backend; TLS termination. Public host: <code>leadmatrix.discovertechnologies.co</code> (also <code>crm.aradiscoveries.com</code> allow-listed).
Application	Single Node.js process managed by <code>systemd</code> unit <code>ara-crm</code> , listening on port 5000. Hosts the API and the in-process cron scheduler. (A managed Azure App Service endpoint also exists as an alternate/observed target.)
Database	Azure Cosmos DB for MongoDB vCore — TLS, SCRAM-SHA-256 auth, <code>retrywrites=false</code> , shared per-cluster connection ceiling (pool sized small, default 5/process).
External	Outbound HTTPS to Meta, Google Ads and Gemini; inbound HTTPS webhook from Meta to <code>/api/meta/webhook</code> .

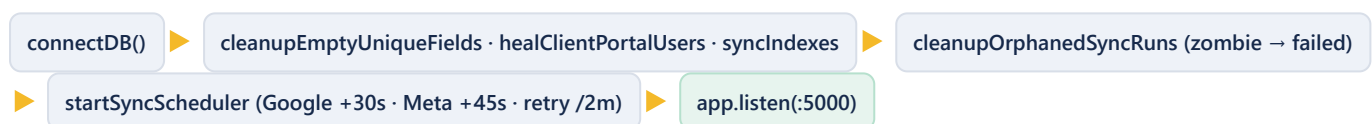
Note. The frontend base URL is environment-driven (`REACT_APP_API_URL`), currently `https://leadmatrix.discovertechnologies.co/api` . The codebase also carries an Azure App Service fallback URL, indicating the app has run on both a self-managed VM and Azure App Service.

10.1 Deployment procedure (redeploy.sh)

```
1. git pull # ~/ara-crm - fetch latest code
2. sudo systemctl restart ara-crm # restart Node backend (systemd)
3. cd crm-frontend && npm run build # CRA build + post-build obfuscation
4. sudo cp -r build/* /var/www/leadmatrix/ # publish static SPA
5. sudo chown -R www-data:www-data /var/www/leadmatrix
  ✓ Deploy done
```

Aspect	Detail
Trigger	Manual (operator runs <code>redeploy.sh</code>); <code>set -e</code> aborts on first failure.
Backend release	In-place git pull + systemd restart (no blue-green; brief restart gap).
Frontend build	<code>react-scripts build</code> then <code>scripts/obfuscate.js</code> (javascript-obfuscator, light preset — identifier mangling; heavy transforms disabled for React 19/MUI/Recharts safety).
Frontend release	Static files copied to the Nginx web root; ownership handed to <code>www-data</code> .
Boot behaviour	On start: connect DB → self-heal migrations → orphaned-sync cleanup → start scheduler → listen. Fatal boot error exits the process (systemd restarts).

10.2 Startup sequence



Component	Technology	Role
Compute	Linux VM (systemd)	Runs the Node.js backend as service <code>ara-crm</code> .
Web server	Nginx	Static hosting of the SPA + reverse proxy + TLS.
Runtime	Node.js (ESM)	Express API + in-process scheduler.
Database	Azure Cosmos DB for MongoDB vCore	Managed, replicated document store.
DNS / TLS	Public domains + HTTPS	<code>leadmatrix.discovertechnologies.co</code> , <code>crm.aradiscoveries.com</code> .
Process supervision	systemd	Auto-restart on crash/exit.
Secrets	Environment files (<code>.env</code>)	DB URI, JWT/encryption keys, API tokens.

Observation. The infrastructure is a single-VM deployment (no container orchestration, no autoscaling group, no managed secret vault). This is appropriate for the current scale but is the primary constraint for the availability and scalability strategies (§33–§36).

12.1 Backend

Concern	Library
Runtime / framework	Node.js (ESM) · Express 5
ODM / database	Mongoose 9 · MongoDB (Cosmos vCore)
Auth / crypto	jsonwebtoken · bcryptjs · Node crypto (AES-256-CBC)
Security middleware	helmet · cors · express-rate-limit · cookie-parser
Validation	express-validator
Scheduling	node-cron
Integrations	google-ads-api · googleapis · Meta Graph (via fetch) · Gemini (REST)
Files / logging	exceljs · multer · morgan
Dev	nodemon

12.2 Frontend

Concern	Library
Framework	React 19 · react-router-dom 6
State	Redux Toolkit · react-redux · React Context
UI	MUI (Material + Joy + Base + System) · Emotion
Data viz	@mui/x-charts · Recharts
Grids / pickers	@mui/x-data-grid · @mui/x-date-pickers
HTTP / forms	axios · Formik · Yup · date-fns
Export	xlsx · xlsx-js-style · jsPDF · jsPDF-autotable · html2canvas · file-saver
Build / harden	react-scripts (CRA) · javascript-obfuscator

```
New_Ara_CRM/
├── backend/                                # Node.js / Express API + jobs
│   ├── config/db.js                      # Mongo connection + boot self-heal
│   ├── controllers/                     # 12 controllers (request handlers)
│   ├── routes/                          # 18 route groups (/api/*)
│   ├── middleware/                      # auth, permissions, validation, errorHandler
│   ├── models/                          # 24 Mongoose schemas
│   ├── services/                        # Meta/Google/metrics/billing/xlsx/AI
│   ├── sync/                            # scheduler, syncService, metaSyncService
│   ├── lib/                             # telecallerAutomation
│   ├── utils/                           # encryption, tokens, mappers, rate-limit, signature
│   ├── scripts/                         # migrations, smoke tests, backfills
│   ├── seedAdmin.js                     # seed a first admin
│   ├── server.js                        # app bootstrap + middleware wiring
│   └── .env(.example)                   # backend configuration
├── crm-frontend/                         # React SPA
│   ├── public/
│   ├── scripts/obfuscate.js             # post-build hardening
│   └── src/
│       ├── api/                         # axios instances + resource clients
│       ├── pages/                       # 23 routed screens
│       ├── components/                  # dashboard/, telecaller/, shared widgets
│       ├── layouts/                     # MainLayout (agency shell)
│       ├── store/slices/                # Redux Toolkit slices
│       ├── contexts/                    # DataCache, Theme
│       ├── constants/                   # telecallerSheet (enum lockstep)
│       └── utils/                       # ProtectedRoute, exporters
├── docs/                                # BRD, FSD, TSD, integration references
└── redeploy.sh                           # manual deployment script
```

14 Folder Structure (responsibilities)

ARA Lead Matrix CRM — Technical Solution Document · v1.0

CONFIDENTIAL

ARA

Folder	Responsibility
backend/config	DB connection, pool sizing, boot-time self-heal & index repair.
backend/routes	HTTP surface; binds middleware guards to controller actions.
backend/controllers	Request/response orchestration & business logic per resource.
backend/middleware	auth (protect/protectClient/protectAdminOrClient/authorize), permissions (checkPermission/checkRole + maps), validation, errorHandler.
backend/models	Schemas, indexes, virtuals, hooks — the data contract.
backend/services	External API clients, normalisation, metrics, billing, Excel, AI.
backend/sync	Cron scheduler + Google/Meta sync orchestration.
backend/lib	Framework-free domain logic (telecaller automation).
backend/utils	Crypto, JWT, Meta field mapping/rate-limit/signature, validators.
backend/scripts	One-off migrations, backfills, and operational smoke tests.
crm-frontend/src/api	Axios base (interceptors, 401 grace) + per-resource clients.
crm-frontend/src/pages	Route-level screens (agency + portal).
crm-frontend/src/components	Reusable UI (dashboard widgets, telecaller grid, reports).
crm-frontend/src/store	Redux Toolkit store & slices.

Backend modules map route group → controller → service(s) → model(s).

Module	Route	Controller / Service	Primary models
Auth	/auth, /client-portal	authController, clientAuthController	User, ClientPortalUser, Client
Users	/users	userController	User
Clients	/clients	clientController	Client (+cascade)
Leads	/leads	leadController + metaMetricsService	Lead
Meta / Telecalling	/meta	metaController, leadTelecallerController, metaLeadService, metaAdsService, metaBillingService, telecallerAutomation, leadXlsxService	Meta*, Lead, BillingTransaction
Google Ads	/google-ads, /metrics, /analytics	googleAdsService, syncService, analyticsService	Campaign, Metric, Keyword
Daily data	/daily-entries, /daily- lead-data	dailyEntryController, dailyLeadDataController	DailyEntry, DailyLeadData
Billing	/payments, /billing	payments/billing routes + metaBillingService	Payment, BillingTransaction, DailyDebitSnapshot
Reports	/reports	reportsController	various (aggregations)
Content	/content-entries	contentEntryController	ContentEntry
Vaults	/vault, /personal-vault	route handlers + encryption util	Vault, PersonalVault
AI	/ai	aiController	— (stateless)

16.1 Backend components

Component	Responsibility
Metrics service	Single authority for lead/spend KPIs (form vs WhatsApp rules, CPL/CTR/CPC/CPM, ingestion gap).
Meta ad service	Graph API client: retries, pagination, rate-limit parsing, field sets.
Meta lead service	Idempotent ingest, retry queue processing, poller.
Billing services	Delta reconcile, idempotent ledger writes, balance cache.
Telecaller automation	Framework-free pre-save rules (reminder/history/DARMANT/stamps).
Xlsx service	Import parse + value cleaning + upsert; export 26-column workbook.
Scheduler	Cron registration, overlap guards, run-state, retry worker.

16.2 Frontend components

Component group	Members
Shell	MainLayout, ProtectedRoute, OfflineScreen, LeadMatrixLoader.
Dashboard	BalanceWatch, PerformanceInsights, KpiStrip, PlatformOverview, ClientOverviewCards, SideWidgets, ActiveCampaigns.
Telecalling	TelecallerSheet, QueueChips, TelecallerFilterBar, ImportLeadsDialog, MetaLeadsTable.
Reporting	TelecallingReport, MonthlyAbstract, MetricsBand, PortalDashboardToday, AiCampaignInsights.
Alerts	LowBalanceBell.

Layer	Contains	Rules
Routing	Express routers	Only wire middleware + controller; no business logic.
Middleware (cross-cut)	auth, permissions, validation, rate-limit, error handler	Guard, validate, normalise; short-circuit on failure.
Controller (domain)	Per-resource handlers	Orchestrate; may call services + models; own transaction-like flows.
Service (integration/domain)	External clients, metrics, billing, xlsx, automation	All external I/O and heavy computation; reusable across controllers.
Data access (ODM)	Mongoose models	Schema, indexes, hooks, virtuals; the only DB touch-point.

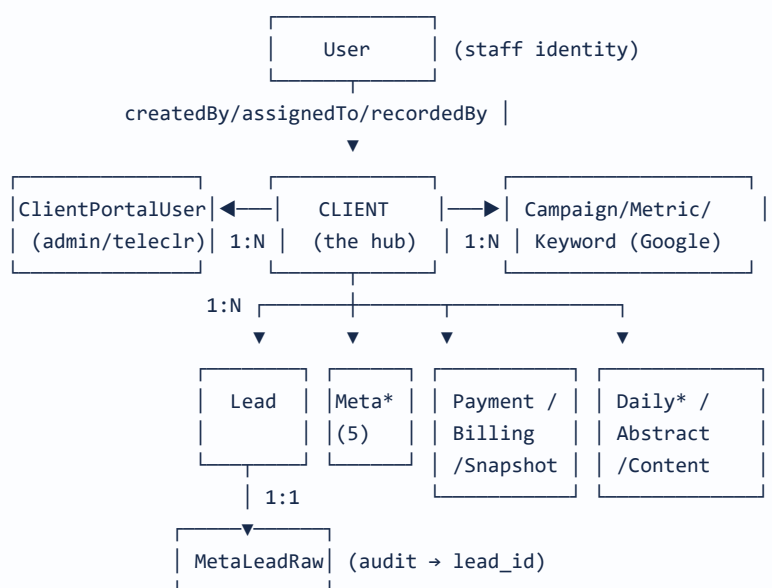
Dependency rule. Dependencies point downward only. Controllers depend on services and models; services depend on models and utils; models depend on nothing above them. The single metrics service is shared by multiple controllers to prevent KPI drift.

Aspect	Detail
Engine	MongoDB API on Azure Cosmos DB for MongoDB vCore (document model).
ODM	Mongoose 9 — schemas, validation, hooks, virtuals, index sync.
Connection	TLS + SCRAM-SHA-256; <code>retrywrites=false</code> ; pool <code>maxPoolSize</code> default 5 (env <code>DB_MAX_POOL</code>), <code>minPoolSize</code> 0, idle 120 s.
Indexing	<code>autoIndex</code> enabled in non-prod only; prod relies on explicit <code>syncIndexes()</code> ; unique/compound/partial indexes enforce integrity & idempotency.
Multi-tenancy	Row-level via <code>client</code> / <code>client_id</code> foreign keys on shared collections.
Integrity patterns	Idempotency keys, partial unique indexes (non-empty), compound daily keys <code>{client, date, ...}</code> , immutable audit/ledger collections.
Self-heal	Boot-time cleanup of empty unique fields, stale portal-user indexes, and orphaned sync runs.

18.1 Collection groups

Group	Collections
Identity	User, ClientPortalUser
Core	Client, Lead
Google Ads	Campaign, Metric, Keyword
Meta Ads	MetaCampaign, MetaAdSet, MetaAd, MetaInsights, MetaLeadForm, MetaLeadRaw, MetaWebhookRetry, MetaSyncRun
Finance	Payment, BillingTransaction, DailyDebitSnapshot
Operational	DailyEntry, DailyLeadData, AbstractEntry, ContentEntry
Secrets	Vault, PersonalVault

CLIENT-CENTRED ER (HUB-AND-SPOKE)



Client is the operational hub (parent of ~15 collections); User is the identity hub. The Meta ad hierarchy (MetaCampaign → MetaAdSet → MetaAd → MetaInsights) is linked by business string IDs, each also referencing Client. MetaLeadRaw links 1:1 to the created Lead via **lead_id**.

19.1 Cardinality summary

Relationship	Cardinality
Client → ClientPortalUser	1 : N
Client → Lead / Meta* / Google* / Finance / Daily*	1 : N (each)
User → Lead (assignedTo, createdBy)	1 : N
User ↔ PersonalVault.sharedWith	M : N
Payment → BillingTransaction (reference.payment_id)	1 : 1
MetaLeadRaw → Lead (lead_id)	1 : 1
ClientPortalUser → ClientPortalUser (createdBy)	self-ref

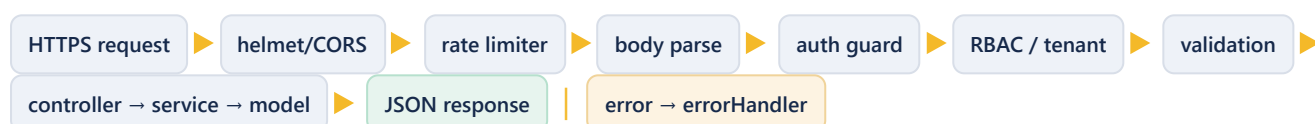
Collection	Key fields	Notable indexes / constraints
User	userID(unique), email(unique), password, role, permissions[], isActive	unique email/userID; role, isActive indexes; bcrypt hook; USRnnn generator.
Client	clientName, status, google_ads_customer_id, meta_ad_account_id, billing{}, meta_pages[], telecalling_targets{}, portal*	unique+sparse on ad IDs; text index; validators (10-digit / act_).
ClientPortalUser	clientId, username, email, password, role, isActive	unique {clientId,username}; partial unique {clientId,email}.
Lead	name, email, contact, source*, status*, meta_*, follow_ups[], telecaller cols	unique {client,contact} partial; unique sparse meta_leadgen_id; many query indexes; virtuals.
Campaign / Metric / Keyword	client_id, campaign_id, metrics/KPIs, date	unique campaign_id; compound {client,campaign,date}; unique keyword {client,campaign,criterion,date}.
MetaCampaign/AdSet/Ad	client_id, *_id, status, budgets, targeting	unique on entity id; {client,status}/{client,campaign} indexes.
MetaInsights	client_id, level, entity_id, date, spend, leads, KPIs	unique {client,entity,date}; {date,level} cross-client.
MetaLeadForm	form_id, page_id, client_id(nullable), question_schema[]	unique form_id; nullable client for unmapped forms.
MetaLeadRaw	leadgen_id(unique), raw_payload, processed, lead_id	unique leadgen_id; {processed, received_at}.
MetaWebhookRetry	leadgen_id, payload, attempts, next_attempt_at, status	{status, next_attempt_at} worker query.
MetaSyncRun	run_id(unique), scope, status, counts{}, errors[]	unique run_id; {status, started_at}.
Payment	client_id, amount, method, date	client_id, date indexes.
BillingTransaction	client_id, type, amount, balance_after, source, idempotency_key	{client,occurred_at}; unique sparse idempotency_key.
DailyDebitSnapshot	client_id, platform, campaign_id, date, debited/reported	unique {client,platform,campaign,date}.
DailyEntry / DailyLeadData	date, client, meta/google counts, CPL	unique {date,client} / unique date; pre-save calc.
AbstractEntry	client, date(YYYY-MM-DD), manualValues(Map)	unique {client,date}.
ContentEntry	date, clientName, contentType, status, approvalStatus	date/clientName/status indexes.
Vault / PersonalVault	credentials (AES), platform, sharedWith[]	encrypted password at rest.

Type	Relationships
ObjectId references (enforced by app)	Lead.client→Client, Lead.assignedTo/createdBy→User, ClientPortalUser.clientId→Client, Payment.client_id→Client, BillingTransaction.client_id→Client & reference.payment_id→Payment, MetaLeadRaw.lead_id→Lead, all Google/Meta facts.client_id→Client, PersonalVault.createdBy/sharedWith→User, AbstractEntry.client→Client, MetaSyncRun.client_id→Client.
String / business-key links	Meta hierarchy cross-links (campaign_id/adset_id/ad_id); Vault.clientId (string); DailyEntry.clientId (string); ContentEntry.clientName (string).
Non-client-scoped	DailyLeadData is a global per-day aggregate (unique on date).
Financial flow	Payment (credit) → BillingTransaction → Client.billing.available_balance; ad spend (Metric/MetaInsights) → delta → DailyDebitSnapshot → BillingTransaction (debit).

Referential integrity is application-enforced (document DB). Hard-delete of a client cascades explicitly across ~10 collections with per-collection outcome reporting; drop is a reversible soft-delete preserving all references.

Aspect	Detail
Style	RESTful JSON over HTTPS, mounted under /api .
Envelope	Success { success:true, data message ... } ; error { success:false, error message errors } .
Auth transport	Authorization: Bearer or httpOnly token cookie (Secure; SameSite=None).
Guards	protect · protectClient · protectAdminOrClient · authorize(roles) · requireClientPortalRole · checkPermission · checkRole · requireAgencyToken.
Rate limiting	Auth 10/15 min (reset on success); API 1000/15 min per IP.
Body limits	10 MB JSON/urlencoded; raw body captured only for the Meta webhook (HMAC).
Caching	Short-TTL in-memory caches on hot read endpoints (analytics 5 min; lead pivots 60 s; user cache 1 min).
Errors	Centralised handler maps Mongoose/JWT errors to status + message.

22.1 Request lifecycle



Group	Guard	Representative endpoints
/auth	public/protect	POST login · POST client-login · POST register · GET me · GET client-me · POST logout · PUT update-details · PUT update-password
/users	admin/superadmin	GET / · POST / · GET stats · GET roles · GET teams · GET/PUT/DELETE :id · PATCH toggle-status · PATCH change-password · PATCH permissions
/clients	checkPermission	GET / · POST / · GET stats · GET/PUT/DELETE :id · PATCH status · PATCH drop · PATCH reonboard · GET :id/portal-credentials · GET smm-users
/leads	checkPermission	GET / · POST / · GET stats/summary · GET monthly-meta-by-client · GET daily-by-client · GET/PUT/DELETE :id
/meta	protectAdminOrClient	GET/POST webhook · sync(+historical) · unassigned-forms · forms/:id/assign reprocess · client/:id/config analytics spend-overview · client/:id/leads (GET/POST/PUT/DELETE) · leads/import export · telecalling-report · telecalling-targets · monthly-abstract(+/cell) · sync-runs · retry-queue · raw-leads
/google-ads	admin/superadmin	POST sync · sync/:id · GET campaigns/:id · PUT customer-id enable associate · POST clients/bulk-associate · GET clients/unassociated · POST validate-customer-id · GET sync-status
/metrics	protectAdminOrClient	GET dashboard/:id · campaigns/:id · daily/:id
/analytics	protectAdminOrClient	GET clients · client/:id · client/:id/spend-overview
/daily-entries	checkPermission	CRUD · stats/summary today · date/:date · meta-lead meta-fund · sync-meta · main-* proxies
/daily-lead-data	checkPermission	CRUD · stats/summary · stats/campaign-comparison · date/:date
/payments	admin/superadmin	POST / · GET :clientId
/billing	admin/superadmin	POST :id/reconcile · :id/deep-sync · :id/reset(confirm)
/client-portal	protectClient	GET analytics · GET/POST/PUT/DELETE users (admin)
/reports	report:view	GET dashboard · sales · campaign-performance · lead-performance · user-performance* · financial* (*admin)
/content-entries	checkPermission	CRUD · calendar/:year/:month · smm-users
/vault · /personal-vault	protect / admin	credential CRUD (decrypt on read); personal share PATCH
/ai	protect	POST analyze-campaign
/health · /	public	health check & root banner

24 Authentication Architecture

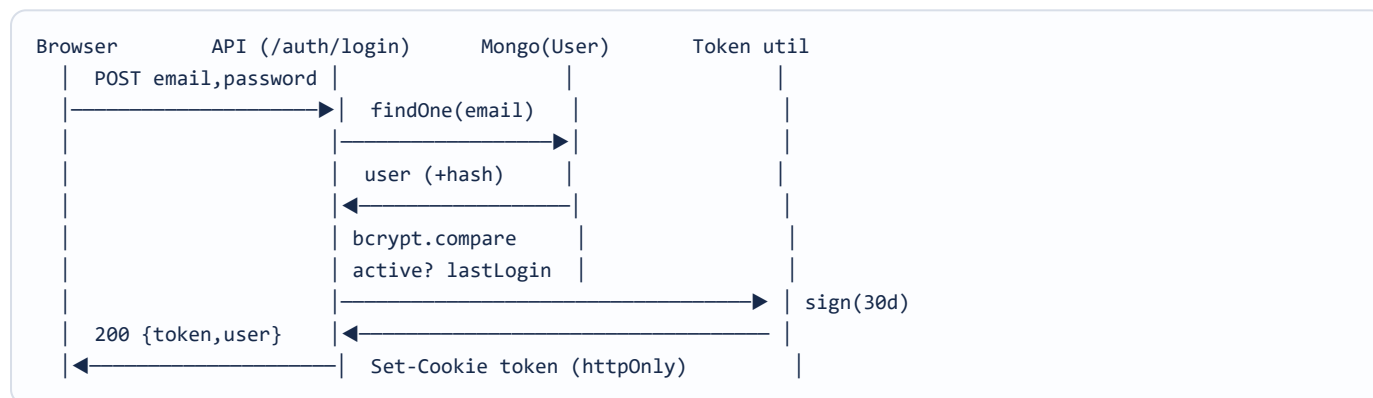
ARA Lead Matrix CRM — Technical Solution Document - v1.0

CONFIDENTIAL

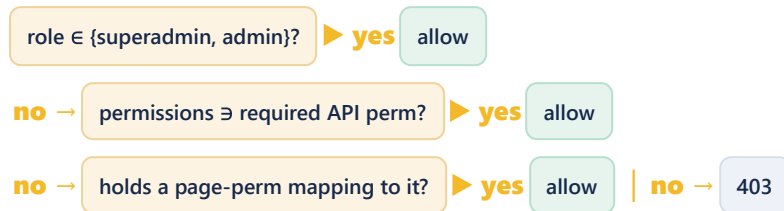
ARA

Aspect	Detail
Mechanism	Stateless JWT (jsonwebtoken), signed with <code>JWT_SECRET</code> .
Staff token	Payload <code>{ id }</code> ; login by email+password; access 30 d, refresh 90 d (<code>JWT_REFRESH_SECRET</code>).
Portal token	Payload <code>{ id, role, clientId, userId }</code> ; login by username/email+password, client-scoped.
Transport	Bearer header or httpOnly cookie <code>token</code> (<code>Secure; SameSite=None</code> for cross-site HTTPS).
Password storage	bcrypt (salt rounds 10) in pre-save hooks; <code>select:false</code> .
Guards	<code>protect</code> (staff; 1-min user cache; active check), <code>protectClient</code> (portal; portalEnabled + active), <code>protectAdminOrClient</code> (dual).
Legacy self-heal	Portal login falls back to legacy client credentials and auto-seeds an admin <code>ClientPortalUser</code> ; tokens without <code>userId</code> degrade to least-privilege on protected portal routes.
Client 401 grace	Front-end delays forced logout 800 ms and cancels it if any success lands — resists transient 401s.

24.1 Login sequence



25.1 Agency RBAC — evaluation order



Page→API map (e.g. page `daily-entry` / `daily-lead-data` grants `entry:create` ; page `clients` / `dashboard` grants `client:read`). Roles are free-form; only superadmin/admin are hard-coded full access.

25.2 Portal RBAC & tenancy

Control	Behaviour
Role gate	<code>requireClientPortalRole('admin')</code> on user-management & ad/billing routes; telecaller limited to leads/reports.
Tenant isolation	<code>clientId</code> taken from the JWT; shared routes reject mismatched <code>:clientId</code> (403 "Portal token cannot access another client").
Last-admin guard	Cannot demote/deactivate/delete the final active admin; cannot delete self.

Gap. `/api/vault` has authentication but no role/permission gate and returns decrypted client credentials to any authenticated staff user (see §26, §50, §53).

Control	Implementation
Transport security	HTTPS end-to-end; secure cookies; HSTS via edge.
HTTP hardening	helmet (cross-origin resource policy set for asset sharing).
CORS	Credentialed, explicit origin allow-list (prod domains + localhost).
Rate limiting	Strict login limiter + generous API limiter.
Password security	bcrypt (rounds 10); never returned; <code>select:false</code> .
Secret encryption	AES-256-CBC (random IV, <code>iv:ciphertext</code>) for page tokens, vault, recoverable portal password; SHA-256 hashing helper.
Webhook integrity	HMAC-SHA256 over raw body vs <code>X-Hub-Signature-256</code> ; verify-token challenge.
Injection resistance	Mongoose parameterised queries; express-validator input validation; enum allow-lists.
Tenant isolation	Token-derived client identity; per-route scoping.
Client hardening	Production bundle obfuscation (identifier mangling).

26.1 Threats & mitigations

Threat	Mitigation
Credential brute force	Auth rate limiter (10/15 min).
Cross-tenant access (IDOR)	JWT-derived tenant + per-route assert.
Token theft	httpOnly secure cookies; short-lived intent (30 d) — see §53 for rotation.
Webhook spoofing	HMAC signature verification.
Secret exposure	Encryption at rest; keys in env (see §53 for a vault recommendation).

27 Logging Strategy

ARA Lead Matrix CRM — Technical Solution Document - v1.0

CONFIDENTIAL

ARA

Log	Detail
HTTP access	morgan — combined in production, dev otherwise.
Application logs	Structured, tagged console logs ([sync-scheduler] , [meta-scheduler] , [meta-retry] , [client-login] , [server]) captured by systemd/journald.
Sync audit	MetaSyncRun documents per run (counts, per-stage errors, rate-limit usage).
Lead audit	MetaLeadRaw immutable payload log; MetaWebhookRetry failure trail.
Financial audit	Append-only BillingTransaction ledger with idempotency keys.
Sensitive data	Login diagnostics never log passwords; errors omit stack traces in production.

Recommendation. Introduce centralised, structured JSON logging (e.g. pino) shipped to an aggregator for search, retention and alerting (§53).

Signal	Source
Liveness	GET /health (uptime, timestamp) and GET / banner.
Sync health	GET /api/meta/sync-status , /sync-runs , /google-ads/sync-status (last run, duration, status, counts, errors).
Retry backlog	GET /api/meta/retry-queue (pending/abandoned).
Ingestion gap	Lead pivots expose missingFormLeads (Meta-reported minus CRM-held).
Balance health	Low-balance bell + Balance Watch as operational monitors.
Process	systemd status / journald for crashes & restarts.

Recommendation. Add uptime checks on /health , alerting on failed/partial sync runs and growing retry backlog, and DB/host metrics (§53).

Condition	Status	Message
CastError (bad ObjectId)	404	"Resource not found"
Duplicate key (11000)	400	"<field> already exists"
ValidationError	400	joined field messages
JsonWebTokenError	401	"Invalid token"
TokenExpiredError	401	"Token expired"
Permission/role denial	403	"Access denied. Required permission/role: ..."
Tenant violation	403	"Portal token cannot access another client"
Unknown route	404	"Not Found - <url>"
Rate limit	429	"Too many ..., please try again later."
Fallback	500	"Server Error" (stack in dev only)

All async handlers are wrapped (`asyncHandler`) so rejections route to the central handler. Process-level `unhandledRejection` / `uncaughtException` handlers log and exit for a clean systemd restart.

Consistency note. Three error-payload key conventions coexist (`message` / `error` / `errors`) and duplicate conflicts return 409 (portal users) vs 400 (staff users); 422 is unused. Standardisation recommended (§53).

THREE-LAYER VALIDATION

L1 Client (Formik/Yup + inline regex)
— instant feedback



L2 API (express-validator + controller
enum checks) — 400 {errors[]}



L3 Schema (Mongoose
validators/enums/indexes) — final
guard

Rule class	Examples
Format	email regex; Google ID 10 digits (schema) / 8–12 (UI); Meta ID <code>act_\d+</code> ; username <code>^[a-z0-9._-]{3,60}\$</code> .
Range/length	password ≥ 6 ; client name 2–200; notes ≤ 1000 ; pagination limit ≤ 10000 ; date range ≤ 90 days.
Enum allow-lists	status, source, telecalling vocabulary (validated by controller before save).
Uniqueness	email, userID, ad IDs (sparse), contact per client (partial), leadgen_id, idempotency key.
Business guards	billing fields stripped from client update; last-admin protection; confirm-required destructive ops.

Technique	Detail
Targeted indexing	Query-shaped indexes on hot paths (client+date, meta_created_time, status, form/campaign/ad ids).
Bulk endpoints	dashboard-overview & daily-metrics replace N+1 per-client calls.
Parallelism	Analytics computed in parallelised Promise.all batches; webhook events processed via Promise.allSettled.
Short-TTL caching	Analytics 5 min, lead pivots 60 s, user lookups 1 min.
Aggregation at DB	Roll-ups computed via Mongo aggregation, not in-app loops.
Field projection	Meta field sets avoid <code>fields=*</code> ; select only what's needed.
Front-end	Route-level code splitting + prefetch; silent background refresh; memoised computed views; balance cache for fast reads.
Sync hygiene	Sequential per-client sync, staggered cadence, overlap guards to avoid contention.

Cache	Scope / TTL	Purpose
Analytics cache	In-process · 5 min	Expensive per-client Google analytics.
Lead-pivot cache	In-process · 60 s	monthly-meta-by-client pivots.
User cache	In-process · 1 min	Auth middleware user lookups.
Balance cache	Client.meta_ad_account_balance	Fast dashboard balance reads (scrapped during sync).
Front-end caches	DataCacheContext + 5-min client-ad cache	Avoid re-fetching slow-changing data across pages.
Main-API proxy cache	5 min	Companion-service clients/leads/funds lookups.

Note. Caches are per-process and in-memory — correct for a single instance. Horizontal scaling would require a shared cache (Redis) to stay consistent (§33).

33 Scalability Strategy

ARA Lead Matrix CRM — Technical Solution Document - v1.0

CONFIDENTIAL

ARA

Dimension	Current & path forward
API tier	Stateless JWT enables horizontal scale-out behind a load balancer. Caveat: in-memory caches and the in-process scheduler assume a single instance.
Scheduler	Must run on exactly one node. To scale the API, extract sync/webhook workers into a dedicated job process (or leader-elect) so only one runs cron.
Database	Cosmos vCore scales vertically/with shards; connection pools kept small due to a hard per-cluster ceiling — more app instances need pool budgeting.
Caching	Move to Redis for a shared cache across instances.
Rate limiting	express-rate-limit is per-process/in-memory; a shared store (Redis) needed for multi-instance correctness.
Vertical growth	Sequential per-client sync scales linearly with client count; parallelise with bounded concurrency + queue as the roster grows.

34 High Availability Strategy

ARA Lead Matrix CRM — Technical Solution Document · v1.0

CONFIDENTIAL

ARA

Layer	Current posture	Recommended
Application	Single VM + systemd auto-restart on crash.	≥2 instances behind a load balancer; health-based routing.
Database	Managed Cosmos DB (built-in replication/HA).	Confirm zone-redundant tier + failover SLA.
Web/edge	Single Nginx.	Redundant edge / managed CDN for static SPA.
Jobs	In-process scheduler (single point).	Dedicated worker with restart + leader election.
Resilience built-in	Overlap guards, zombie-run cleanup, retry queue, poller safety-net, graceful degradation.	—

Current SPOF. The single application VM is the main availability risk; the database tier is managed and already highly available.

35 Backup Strategy

Asset	Approach
Database	Azure Cosmos DB automatic backups (platform-managed, point-in-time depending on tier). Action: confirm retention window & restore runbook.
Application code	Git repository is the source of truth; deploy is a git pull.
Configuration/secrets	<code>.env</code> files on the host — not in version control; must be backed up securely out-of-band.
Immutable audit	Ledger + raw-lead + sync-run collections provide replay/audit even without file backups.
User-generated exports	Excel/PDF generated on demand — no separate backup needed.

Recommendation. Document and test DB restore; store `.env` secrets in a managed vault with backup; schedule periodic logical exports for an independent recovery copy (§53).

Scenario	Recovery approach
Application host loss	Re-provision VM, install Node/Nginx, restore <code>.env</code> , <code>git pull</code> , <code>redploy.sh</code> . Boot self-heal repairs indexes; scheduler resumes; retry queue & poller recover missed leads.
Database loss/corruption	Restore from Cosmos backup to last good point; the ledger/raw-lead audit enables reconciliation of any gap.
Integration outage	Sync skips/back off; leads queue in retry; balances resume on recovery — no data loss.
Secret compromise	Rotate JWT/encryption keys & API tokens; re-encrypt secrets; force re-login.

Objective	Indicative target (to formalise)
RTO (app)	Hours — bounded by VM re-provision + deploy.
RPO (data)	Minutes-to-hours — bounded by Cosmos backup granularity; near-zero for leads/billing thanks to idempotent replay.

Recommendation. Formalise RTO/RPO, automate infrastructure provisioning (IaC), and rehearse a restore drill (§53).

Stage	Current state
Source control	Git (GitHub) — single <code>main</code> branch workflow observed.
Continuous integration	None — no pipeline (<code>.github/workflows</code> absent); no automated build/test gate.
Build	Manual: <code>npm run build</code> (CRA + obfuscation) at deploy time.
Deployment	Manual <code>redeploy.sh</code> : <code>git pull</code> → <code>systemd restart</code> → <code>build</code> → <code>publish static</code> .
Rollback	Git revert + re-run deploy (no automated rollback).
Migrations	Idempotent, boot-time self-heal + manual scripts under <code>backend/scripts</code> .

Recommendation. Introduce a CI pipeline (lint + test + build on PR) and a CD step (artifact build, blue-green or rolling deploy, automated rollback) — see §53.

Variable	Purpose
NODE_ENV / PORT	Runtime mode; API port (5000).
MONGODB_URI	Cosmos vCore connection string (TLS, SCRAM).
DB_MAX_POOL	Per-process connection pool cap (default 5).
JWT_SECRET / JWT_EXPIRE	Access-token signing & lifetime (30 d).
JWT_REFRESH_SECRET / JWT_REFRESH_EXPIRE	Refresh-token signing & lifetime (90 d).
COOKIE_EXPIRE	Auth cookie lifetime (days).
ENCRYPTION_KEY	AES-256 key (padded/truncated to 32 bytes).
CLIENT_URL	Allowed CORS origin (prod frontend).
RATE_LIMIT_* / AUTH_RATE_LIMIT_MAX	Rate-limit windows & ceilings.
SYNC_INTERVAL_MINUTES	Google sync cadence (15).
META_APP_ID / SECRET / VERIFY_TOKEN / SYSTEM_USER_TOKEN	Meta app + webhook + system-user auth.
META_API_VERSION / BASE_URL	Graph API version pin (v19.0) & host.
META_SYNC_ENABLED / _INTERVAL / _BACKFILL_DAYS	Meta sync toggle, cadence, backfill window.
META_BILLING_DRY_RUN	Financial safety switch (default true).
GOOGLE_ADS_* (OAuth, dev token, MCC)	Google Ads API credentials.
GEMINI_API_KEY / GEMINI_MODEL	AI insights (default gemini-1.5-flash).
MAIN_API_URL / META_SYNC_API_URL	Companion-service endpoints (Daily Entry).
REACT_APP_API_URL (frontend)	API base URL for the SPA.

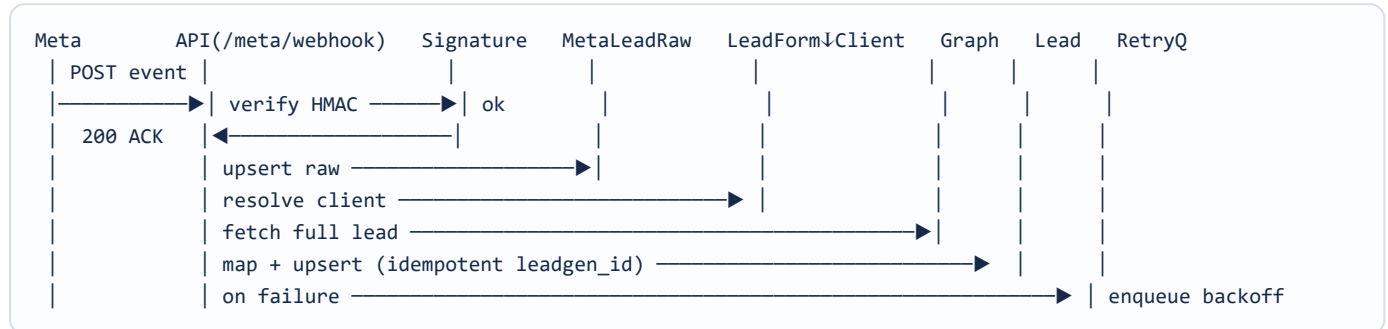
File	Purpose
backend/.env(.example)	Backend runtime configuration & secrets.
crm-frontend/.env	Frontend build-time config (REACT_APP_API_URL).
backend/package.json	Scripts (start/dev/seed/fix:indexes/oauth), dependencies.
crm-frontend/package.json	Scripts (start/build/build:raw/obfuscate/test), deps, browserslist, eslintConfig.
backend/config/db.js	Connection, pool sizing, boot self-heal.
backend/server.js	Middleware wiring, CORS allow-list, rate limits, route mounting, boot sequence.
crm-frontend/scripts/obfuscate.js	Post-build hardening configuration.
redeploy.sh	Deployment steps.
systemd unit (ara-crm)	Process supervision (host-side).
Nginx site config	Static host + reverse proxy + TLS (host-side).

Integration	Protocol / auth	Data & direction
Meta Graph API	REST · system-user token + encrypted page tokens · HMAC webhooks	IN: campaigns, adsets, ads, insights, forms, leads. OUT: page subscribe/unsubscribe. Real-time leadgen webhooks.
Google Ads API	gRPC/REST client · OAuth refresh + dev token + MCC	IN: campaigns, daily metrics (incl. impression share, click types), keyword metrics.
Google Gemini	REST · API key	OUT: performance snapshot. IN: strict-JSON insight (grade, strengths, strategies).
Companion "Main API" & Meta-sync API	REST (Azure-hosted)	Daily-Entry module: clients/leads/funds lookups + trigger meta-sync (cached 5 min).

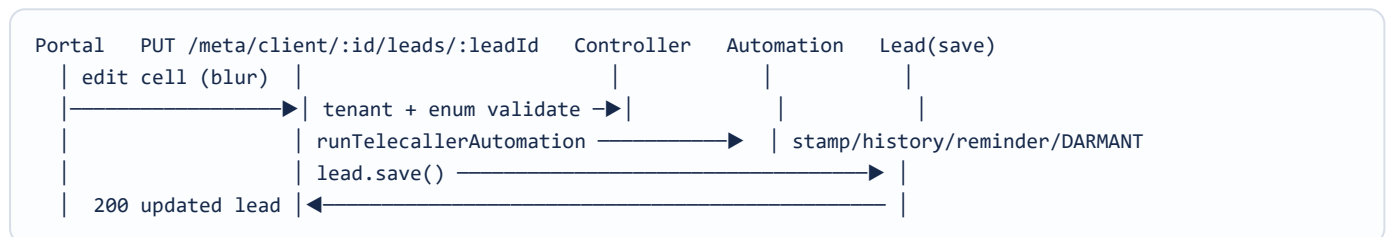
40.1 Integration resilience

- Typed errors (auth/permission/rate-limit/transient/validation) drive stop/skip/backoff/retry decisions.
- Exponential backoff (2s/8s/32s) for transient failures; cursor pagination capped ($\leq 10k/\text{edge}$).
- Rate-limit header parsing ($\geq 80\%$ usage $\rightarrow$ throttle); sequential per-client processing.
- Webhook retry dead-letter queue + independent poller safety-net.
- AI truncated-JSON repair; graceful 503 when the AI key is absent.

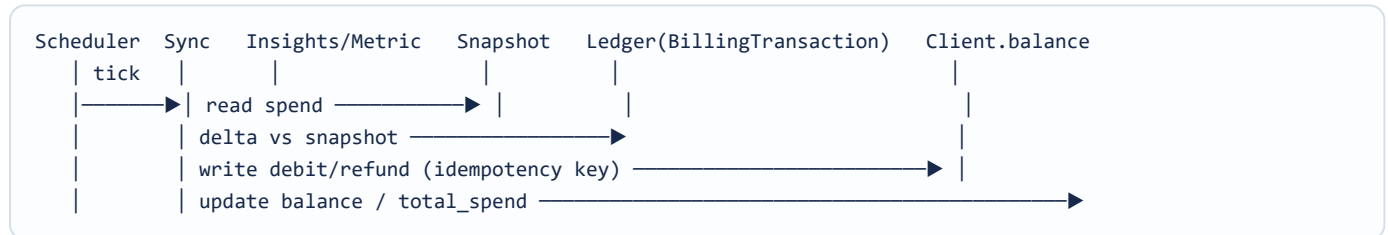
41.1 Meta lead ingestion (webhook)



41.2 Telecaller save with automation



41.3 Billing reconciliation (per campaign/day)



42 Data Flow Diagrams

ARA Lead Matrix CRM — Technical Solution Document · v1.0

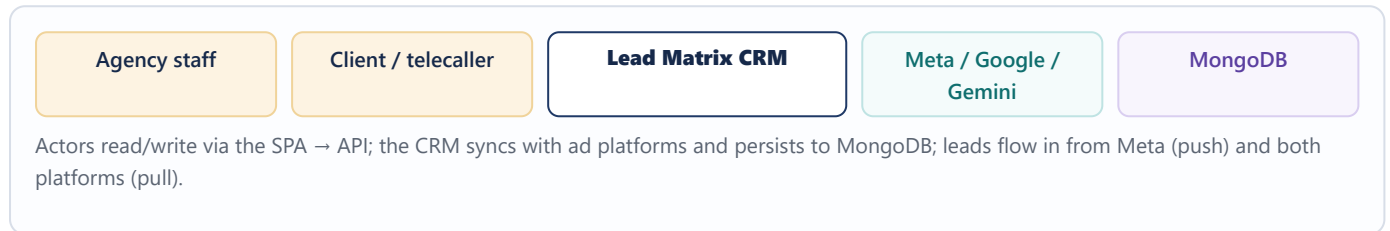
CONFIDENTIAL

ARA

42.1 Ad-spend → lead → conversion → billing



42.2 Level-0 context



42.3 Excel round-trip



BACKEND COMPONENTS & DEPENDENCIES

HTTP SURFACE

Routers

Middleware (auth/permissions/validation/error)



CONTROLLERS

auth

client

lead/telecaller

meta

reports/ai

▼ use

SERVICES

metaAds/metaLead/metaBilling

googleAds/analytics

leadXlsx

telecallerAutomation

▼ persist

MODELS (MONGOOSE)

24 schemas

▼ trigger

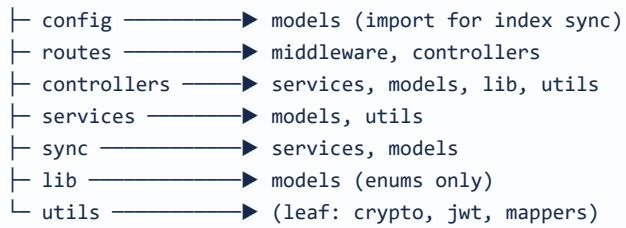
JOBS

scheduler → syncService / metaSyncService / retry worker

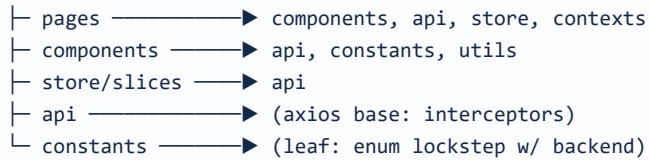


Mongoose "classes" = schemas with instance methods (comparePassword, matchPortalPassword), statics/hooks (pre-save bcrypt/mirror), and virtuals (follow_up_number, reminder_date, month). Sub-documents: follow_ups[], meta_pages[], drop_history[], question_schema[].

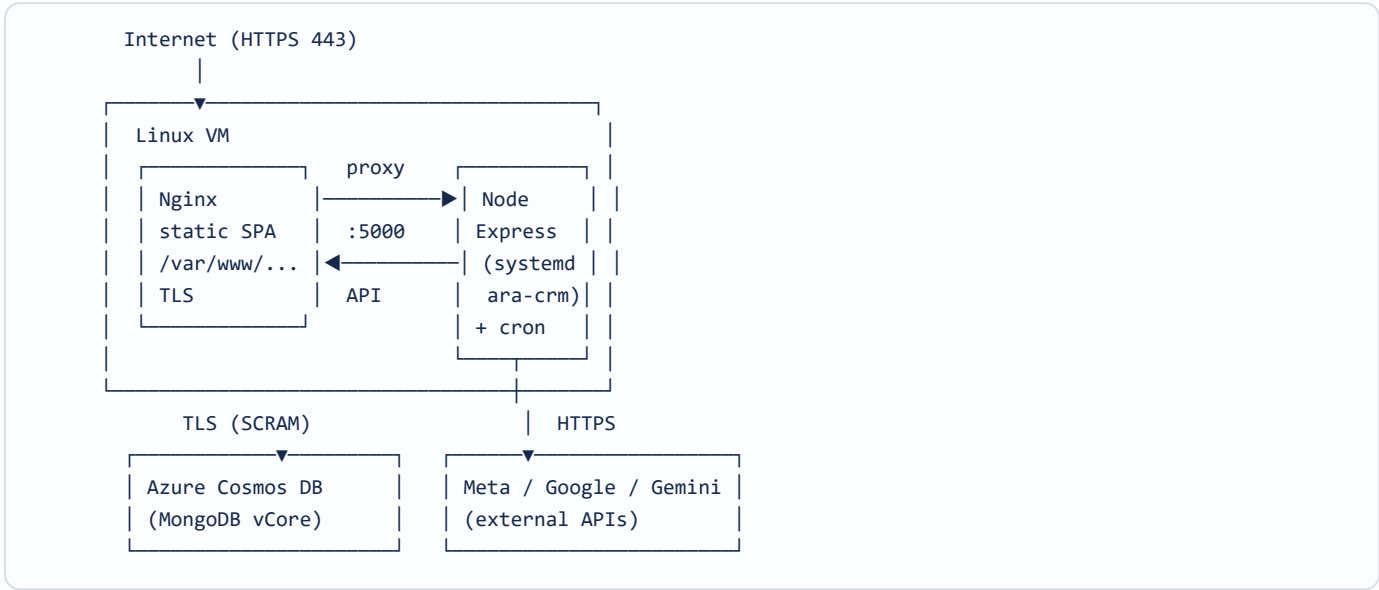
backend (package)



crm-frontend (package)



Package dependencies are acyclic and point toward leaf utilities/models. The frontend `constants/telecallerSheet` intentionally mirrors backend `models/Lead` enums (kept in lockstep).



Artifact	Deployed to
SPA build (obfuscated)	Nginx web root /var/www/leadmatrix .
Node API + scheduler	systemd service <code>ara-crm</code> :5000.
Database	Managed Cosmos DB (external, replicated).

Module	Depends on
server.js	config/db, all routes, sync/scheduler, middleware.
routes/*	middleware (auth/permissions/validation), controllers.
controllers/metaController	metaLeadService, metaAdsService, metaBillingService, metaMetricsService, telecallerAutomation, leadXlsxService, Meta* models.
sync/scheduler	syncService (Google), metaSyncService, metaLeadService (retry worker).
services/metaMetricsService	MetalInsights, Lead (shared KPI authority; consumed by dashboard, leads, analytics).
services/*billing*	BillingTransaction, DailyDebitSnapshot, Client, utils.
lib/telecallerAutomation	models/Lead enums only (no DB calls — pure functions).
utils/*	Leaf modules (crypto, jwt, field mapper, rate-limit, signature).

Coupling note. `metaController.js` is the largest unit (~2,700 LOC) and the highest-fan-out module; a candidate for decomposition into smaller controllers/services (§53).

Area	Observed convention
Language / modules	Modern JS, ES modules (backend <code>"type": "module"</code>); async/await throughout.
Error propagation	<code>asyncHandler</code> wrapper + centralised handler; custom <code>ErrorResponse</code> .
Naming	camelCase JS; snake_case for ad-platform-sourced fields (mirror external shapes); PascalCase models.
Structure	Strict routes→controllers→services→models layering; framework-free domain logic in <code>lib/</code> .
Documentation	Extensive intent-explaining comments (esp. schemas, billing, automation, self-heal) — a strong maintainability asset.
Consistency	Shared enum constants across front/back; single metrics service to prevent KPI drift.
Linting	Frontend: CRA ESLint (react-app + jest). Backend: no enforced linter configured.
Config	Env-driven with sensible defaults; secrets never committed.

Recommendation. Add a shared linter/formatter (ESLint + Prettier) and pre-commit hooks across both packages for uniform style enforcement (§53).

Test type	Current state
Unit tests	Minimal — only the default CRA <code>App.test.js</code> ; no backend unit suite.
Integration tests	None automated.
Operational smoke tests	Manual scripts under <code>backend/scripts</code> (<code>metaSmokeTest</code> , <code>metaSyncSmokeTest</code> , <code>metaWebhookSmokeTest</code> , <code>metaBillingSmokeTest</code> , <code>metaPhase6SmokeTest</code> , <code>testGoogleAds</code>) — run by hand against live integrations.
Test tooling present	@testing-library/react + jest-dom (frontend) available but under-used.

49.1 Recommended test pyramid

Layer	Target
Unit	Pure logic first: <code>telecallerAutomation</code> (reminder/DARMANT/history), metrics computations, validators, encryption.
Integration	API + in-memory Mongo: auth/RBAC, tenant isolation, idempotent ingest, billing reconcile.
Contract	Mock Meta/Google/Gemini responses to test error typing & mapping.
E2E	Critical journeys (login, connect client, work a lead, view report) via Playwright/Cypress.

Ref	Consideration	Status / action
SC-01	Vault authorization gap	Open — add role/permission gate + access audit to <code>/api/vault</code> .
SC-02	Secret management	Improve — move keys/tokens from <code>.env</code> to a managed secret vault; rotate.
SC-03	Token lifetime & rotation	Improve — long-lived access token; wire refresh-exchange & shorten access TTL.
SC-04	Encryption mode	Note — AES-256-CBC (no auth tag); consider AES-GCM for integrity.
SC-05	Distributed rate limiting	Note — move to shared store before multi-instance.
SC-06	Tenant isolation	Strong — token-derived, per-route enforced.
SC-07	Webhook integrity	Strong — HMAC verified.
SC-08	Dependency hygiene	Improve — add automated dependency/vuln scanning in CI.
SC-09	Audit trail (end-user actions)	Improve — add per-field change history beyond financial/lead audit.

51 Technical Risks

ARA Lead Matrix CRM — Technical Solution Document - v1.0

CONFIDENTIAL

ARA

Ref	Risk	Lik.	Imp.	Mitigation
TR-01	Single-VM SPOF (app + scheduler)	Med	High	Multi-instance API + dedicated worker; systemd restart interim.
TR-02	No CI/CD & minimal tests	High	Med	Introduce CI (lint/test/build) + automated deploy; build test pyramid.
TR-03	API token expiry halts sync/billing	Med	High	Typed errors, alerts, token-refresh runbook, graceful UI.
TR-04	Meta API version deprecation	Med	Med	Centralised version/field config; planned upgrades.
TR-05	Cosmos connection-ceiling exhaustion	Low	High	Small pools; budget pools per instance when scaling.
TR-06	Vault credential exposure	Med	High	Add authorization + auditing (SC-01).
TR-07	metaController monolith (~2.7k LOC)	Med	Low	Refactor into focused controllers/services.
TR-08	In-memory cache/rate-limit under scale-out	Low	Med	Shared Redis cache/limiter store.
TR-09	Manual deploy human error	Med	Med	Automate build/deploy with rollback.

Initiative	Benefit
Extract job/worker service	Decouple sync/webhook processing from the API; enables API scale-out and isolates load.
Message queue for ingestion	Buffer webhook/lead spikes; smooth Graph API pressure; durable retries.
Shared cache (Redis)	Consistent caching + distributed rate limiting across instances.
Bounded-concurrency sync	Parallelise per-client sync with a concurrency cap as the roster grows.
Read replicas / sharding	Scale read-heavy analytics; shard by client for very large datasets.
Containerisation + orchestration	Repeatable deploys, autoscaling, rolling updates.
CDN for SPA	Global static delivery + edge caching.

53 Technical Recommendations

ARA Lead Matrix CRM — Technical Solution Document · v1.0

CONFIDENTIAL

ARA

Ref	Recommendation	Priority	Outcome
RC-01	Gate & audit the credential vault	P1	Closes the key authorization gap.
RC-02	Add CI (lint+test+build) & automated CD with rollback	P1	Safer, repeatable releases.
RC-03	Build the test pyramid (start with pure logic)	P1	Regression safety on billing & automation.
RC-04	Run ≥ 2 API instances + extract the scheduler worker	P2	Removes the SPOF; enables scale-out.
RC-05	Managed secret vault + key rotation + AES-GCM	P2	Stronger secret handling & integrity.
RC-06	Refresh-token rotation + shorter access TTL	P2	Reduced token-theft blast radius.
RC-07	Centralised structured logging + monitoring/alerting	P2	Faster incident detection & resolution.
RC-08	Standardise API error envelope & status codes	P3	Predictable client integration.
RC-09	Decompose metaController; introduce Redis cache/limiter	P3	Maintainability & multi-instance readiness.
RC-10	Formalise RTO/RPO, IaC, and a restore drill	P2	Proven disaster recovery.

A • Backend script inventory (operational)

Category	Scripts
Migrations	migrations/2026-04-meta-schema-init, migrateLeadTelecallerFields, fixDailyEntryIndexes, backfillBillingLedger, metaBackfillCreatedTime.
Meta onboarding/backfill	bulkFetchAdAccountMeta, bulkFetchPageTokens, bulkSubscribePages, metaAssignPagesToSystemUser, metaPagesManualLink, metaMappingApply/Report, seedMetaClients.
OAuth / tokens	getRefreshToken, simpleRefreshToken.
Smoke tests	metaSmokeTest, metaSyncSmokeTest, metaWebhookSmokeTest, metaBillingSmokeTest, metaPhase6SmokeTest, testGoogleAds, metaVerifyAccess, inspectMetaActions.
Seed	seedAdmin, seedTeams.

B • Key npm scripts

Script	Action
backend: start / dev	<code>node server.js</code> / <code>nodemon</code> .
backend: seed:admin / fix:indexes	Seed first admin / repair daily-entry indexes.
backend: get-refresh-token / oauth-setup	Google OAuth token acquisition.
frontend: build	<code>react-scripts build</code> + <code>obfuscate.js</code> .
frontend: build:raw / start / test	Un-obfuscated build / dev server / CRA tests.

C • Technology versions (selected)

Component	Version
Express	5.x
Mongoose	9.x
React / react-router	19.x / 6.x
MUI	7.x
Meta Graph API	v19.0 (pinned)
Gemini model	gemini-1.5-flash (default)

D • Related documents

- Business Requirements Document — ARA-BRD-CRM-2026-001
- Functional Specification Document — ARA-FSD-CRM-2026-001
- Meta Ads Integration Plan & API Reference; Google Ads Analytics API reference (docs/).

End of Document. This Technical Solution Document describes the ARA Lead Matrix CRM as built and in production as of 17 July 2026. Version 1.0 (baseline); companion to the BRD and FSD; classified Confidential.